Logbook

Background

I will be conducting my sustainability audit on “Interpot”, which is a biological interpretability project (Adams et al., 2025). As described in their paper, this project is motivated by the goal to identify the features of pLMs (protein learning models) through training sparse autoencoders (SAEs) on the residual stream of a pLM. They uncovered some useful interpretability insights!

1) pLMs use a combination of generic features and family-specific features to represent a protein.
2) known sequence determinants of properties (like thermostability and subcellular localization) can be identified via linear probing of SAE features.

I will be analyzing the carbon footprint of an energy hotspot in their workflow and whether any optimizations can be made for eco-friendlier computing in future pLM interpretability experimentation.

Step 1: Selecting Energy Hotspot Notebook from Interpot Repo

The interpot Github repo contains the following notebooks.

(Note: SAE = Sparse Autoencoder)

Here is an overview of what each does:

- sae_inference → runs SAE inference on protein sequences by loading the pre-trained ESM-2 + SAE from HuggingFace, processes inputs, extracts/interprets sparse latent activations/features.
- sae_test_steering → tests SAE based steering/interventions

- subcellular_localization_linear_probe → trains and evaluates a linear probe (logistic regression) on top of SAE and ESM embeddings to predict subcellular localization from protein sequences.

The activation/latent collection phase (running the ESM-2 + SAE pipeline over labeled proteins to gather features for probing), and then using that to train a linear probe that predicts subcellular localization is the most data-intensive and GPU-bound process in the workflow, with repeated forward passes. I will consider it the main energy hotspot. This can be observed via cursory preview of the notebooks. Due to this reason, I will be analyzing the carbon footprint of this process.

Step 2: Uploading the Notebook on Google Colab and Adding my CodeCarbon Code

(Runtime was connected to T4 GPU, and the notebook was granted secret access to my HuggingFace token.)
First of all, I ran the notebook cells that loaded the primary libraries and dataset. I was encountering a numpy binary incompatibility issue which I handled via uninstalling and reinstalling numpy and pandas, then restarting runtime.

```
numpy/random/mtrand.pyx in init numpy.random.mtrand()

ValueError: numpy.dtype size changed, may indicate binary incompatibility. Expected 96 from C header, got 88 from PyObject
```

This resolved it.

Then the subsequent cells were run. I added my CodeCarbon code to track the carbon footprint of the activation/latent collection process right before its respective cell (the first cell under the “ESM and SAE inference” code block).

Copy of subcellular_localization_linear_probe.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Compute and store mean-pooled embeddings for both ESM and SAE

[5]

```
!pip install codecarbon
```

[6] 3s

```
from codecarbon import EmissionsTracker
import os

#creating folder to save my reslts
os.makedirs("carbon_logs", exist_ok=True)

#creating the main tracker (measures GPU/CPU energy use)
tracker = EmissionsTracker(
    project_name="InterProt_Probe_Activations",
    output_dir="carbon_logs",
    measure_power_secs=15,          #checking power often enough
    log_level="info",               #will show messages in output
    on_csv_write="update"           #re-runs should be fine

)

#start measuring
tracker.start()
print("Codecarbon is now tracking the energy!")
```

Step 3: Running the Notebook with my CodeCarbon Code

As shown in the previous screenshot, the CodeCarbon code was run first to initiate my tracker. Then the probe activation and latent collection cells were run.

Copy of subcellular_localization_linear_probe.ipynb

File Edit View Insert Runtime Tools Help

mands + Code + Text Run all

[7]

```
for seq_idx, row in tqdm(enumerate(df.iter_rows(named=True)), total=len(df)):
    seq = row['sequence']
    esm_layer_acts = get_layer_activations(
        tokenizer=tokenizer, plm=esm_model, seqs=[seq], layer=LAYER
    )[0][1:-1] # Trim off BoS and EoS tokens

    seq_esm_acts[seq] = torch.mean(esm_layer_acts, axis=0)

    sae_acts = sae_model.get_acts(esm_layer_acts)
    seq_sae_acts[seq] = torch.mean(sae_acts, axis=0)
```

```
[codecarbon INFO @ 19:09:44] 0.080262 kWh of electricity and 0.000000 L of water were used since the beginning.
 98%|[redacted]| 10233/10461 [40:56<01:02,  3.63it/s][codecarbon INFO @ 19:09:59] Energy consumed for RAM : 0.006826 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:09:59] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:09:59] Energy consumed for All CPU : 0.029014 kWh
[codecarbon INFO @ 19:09:59] Energy consumed for all GPUs : 0.044915 kWh. Total GPU Power : 65.9791395901296 W
[codecarbon INFO @ 19:09:59] 0.080755 kWh of electricity and 0.000000 L of water were used since the beginning.
 98%|[redacted]| 10295/10461 [41:11<00:50,  3.27it/s][codecarbon INFO @ 19:10:14] Energy consumed for RAM : 0.006868 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:10:14] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:10:14] Energy consumed for All CPU : 0.029191 kWh
[codecarbon INFO @ 19:10:14] Energy consumed for all GPUs : 0.045188 kWh. Total GPU Power : 65.51282467334396 W
[codecarbon INFO @ 19:10:14] 0.081247 kWh of electricity and 0.000000 L of water were used since the beginning.
 99%|[redacted]| 10359/10461 [41:26<00:19,  5.30it/s][codecarbon INFO @ 19:10:29] Energy consumed for RAM : 0.006909 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:10:29] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:10:29] Energy consumed for All CPU : 0.029368 kWh
[codecarbon INFO @ 19:10:29] Energy consumed for all GPUs : 0.045463 kWh. Total GPU Power : 65.9986551915988 W
[codecarbon INFO @ 19:10:29] 0.081740 kWh of electricity and 0.000000 L of water were used since the beginning.
100%|[redacted]| 10431/10461 [41:41<00:05,  5.39it/s][codecarbon INFO @ 19:10:44] Energy consumed for RAM : 0.006951 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:10:44] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:10:44] Energy consumed for All CPU : 0.029545 kWh
[codecarbon INFO @ 19:10:44] Energy consumed for all GPUs : 0.045735 kWh. Total GPU Power : 65.46373721825351 W
[codecarbon INFO @ 19:10:44] 0.082231 kWh of electricity and 0.000000 L of water were used since the beginning.
100%|[redacted]| 10461/10461 [41:48<00:00,  4.17it/s]
```

- Train classifier on ESM embedding (for accuracy comparison)
- Train classifier on SAE embedding
- Inspect and write the weights of the SAE classifier

[8]
✓ 10m

```python
with warnings.catch_warnings():
    warnings.simplefilter("ignore")

    print("Training ESM classifier")
    esm_classifier, _, _ = train_classifier(seq_esm_acts)
    esm_accuracy = evaluate_classifier(esm_classifier, seq_esm_acts)
    print(f"ESM accuracy: {esm_accuracy}")

    print("Training SAE classifier")
    sae_classifier, _, _ = train_classifier(seq_sae_acts)
    sae_accuracy = evaluate_classifier(sae_classifier, seq_sae_acts)
    print(f"SAE accuracy: {sae_accuracy}")
    write_sorted_weights(sae_classifier, 'weights')
```

```
Training ESM classifier
[codecarbon INFO @ 19:10:59] Energy consumed for RAM : 0.006993 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:10:59] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:10:59] Energy consumed for All CPU : 0.029722 kWh
[codecarbon INFO @ 19:10:59] Energy consumed for all GPUs : 0.045936 kWh. Total GPU Power : 48.10512196433605 W
[codecarbon INFO @ 19:10:59] 0.082650 kWh of electricity and 0.000000 L of water were used since the beginning.
[codecarbon INFO @ 19:10:59] 0.004469 g.CO2eq/s mean an estimation of 140.9316709817295 kg.CO2eq/year
[codecarbon INFO @ 19:11:14] Energy consumed for RAM : 0.007034 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:11:14] Delta energy consumed for CPU with constant : 0.000177 kWh, power : 42.5 W
[codecarbon INFO @ 19:11:14] Energy consumed for All CPU : 0.029899 kWh
[codecarbon INFO @ 19:11:14] Energy consumed for all GPUs : 0.046072 kWh. Total GPU Power : 32.62342895984711 W
[codecarbon INFO @ 19:11:14] 0.083005 kWh of electricity and 0.000000 L of water were used since the beginning.
[codecarbon INFO @ 19:11:29] Energy consumed for RAM : 0.007076 kWh. RAM Power : 10.0 W
```

Finally, the tracker was stopped.

[9]
✓ 0s

```python
#Stopping my tracker!
emissions = tracker.stop()
print(f"Emissions from this training run: {emissions:5f} kg CO2eq")
```

(It was ensured that all these cells run sequentially without any idle time by using the “run all” option on the “ESM and SAE inference” and “Linear classifiers” blocks. This prevents CodeCarbon from tracking idle runtime and thus gives an accurate measurement.)

Step 4: Saving Output and Generating Report from GreenMetaData tool

This was the output from CodeCarbon after stopping the tracker.

```
[9]
✓ 0s
#Stopping my tracker!
emissions = tracker.stop()
print(f"Emissions from this training run: {emissions:5f} kg CO2eq")

[codecarbon INFO @ 19:20:54] Energy consumed for RAM : 0.008644 kWh. RAM Power : 10.0 W
[codecarbon INFO @ 19:20:54] Delta energy consumed for CPU with constant : 0.000120 kWh, power : 42.5 W
[codecarbon INFO @ 19:20:54] Energy consumed for All CPU : 0.036742 kWh
[codecarbon INFO @ 19:20:54] Energy consumed for all GPUs : 0.051571 kWh. Total GPU Power : 34.81735160666163 W
[codecarbon INFO @ 19:20:54] 0.096957 kWh of electricity and 0.000000 L of water were used since the beginning.
[codecarbon INFO @ 19:20:54] 0.003356 g.CO2eq/s mean an estimation of 105.8314078289027 kg.CO2eq/year
Emissions from this training run: 0.013453 kg CO2eq
```

Overall Energy Consumption

Energy consumed for RAM : 0.008644 kWh. RAM Power : 10.0 W
Delta energy consumed for CPU with constant : 0.000120 kWh, power : 42.5 W
Energy consumed for All CPU : 0.036742 kWh
Energy consumed for all GPUs : 0.051571 kWh. Total GPU Power : 34.81735160666163 W
0.096957 kWh of electricity and 0.000000 L of water were used since the beginning.
0.003356 g.CO2eq/s mean an estimation of 105.8314078289027 kg.CO2eq/year
Emissions from this training run: 0.013453 kg CO2eq

Energy Consumption Breakdown

Furthermore, the process-wise energy consumption breakdown (screenshots attached above and can also be viewed in notebook outputs), can be summarized as:

Process	CPU Energy Consumed	GPUs Energy Consumed	Total GPU Power	Electricity Used	Water Used
ESM & SAE Inference	0.029545 kWh	0.045735 kWh	65.46373721825351 W	0.082231 kWh	0.000 L
Linear Classifiers	0.036622 kWh	0.036622 kWh	34.46987631706962 W	0.096710 kWh	0.000 L

Next, the GreenMetaData tool was run through my command prompt after downloading the zipped folder from their repository.

```
Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\POSH TOUCH\Desktop\GreenMetaData> python3 -m venv .venv
PS C:\Users\POSH TOUCH\Desktop\GreenMetaData> .venv\Scripts\Activate.ps1
```

```
PS C:\Users\POSH TOUCH\Desktop\GreenMetaData> .venv\Scripts\activate.bat
PS C:\Users\POSH TOUCH\Desktop\GreenMetaData> pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting asgiref==3.8.1 (from -r requirements.txt (line 1))
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting Django==5.2.1 (from -r requirements.txt (line 2))
  Downloading django-5.2.1-py3-none-any.whl.metadata (4.1 kB)
Collecting PyYAML==6.0.2 (from -r requirements.txt (line 3))
  Downloading PyYAML-6.0.2-cp313-cp313-win_amd64.whl.metadata (2.1 kB)
Collecting sqlparse==0.5.3 (from -r requirements.txt (line 4))
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from Django==5.2.1->-r requirements.txt (line 2))
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading asgiref-3.8.1-py3-none-any.whl (23 kB)
Downloading django-5.2.1-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 12.5 MB/s  0:00:00
Downloading PyYAML-6.0.2-cp313-cp313-win_amd64.whl (156 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
```

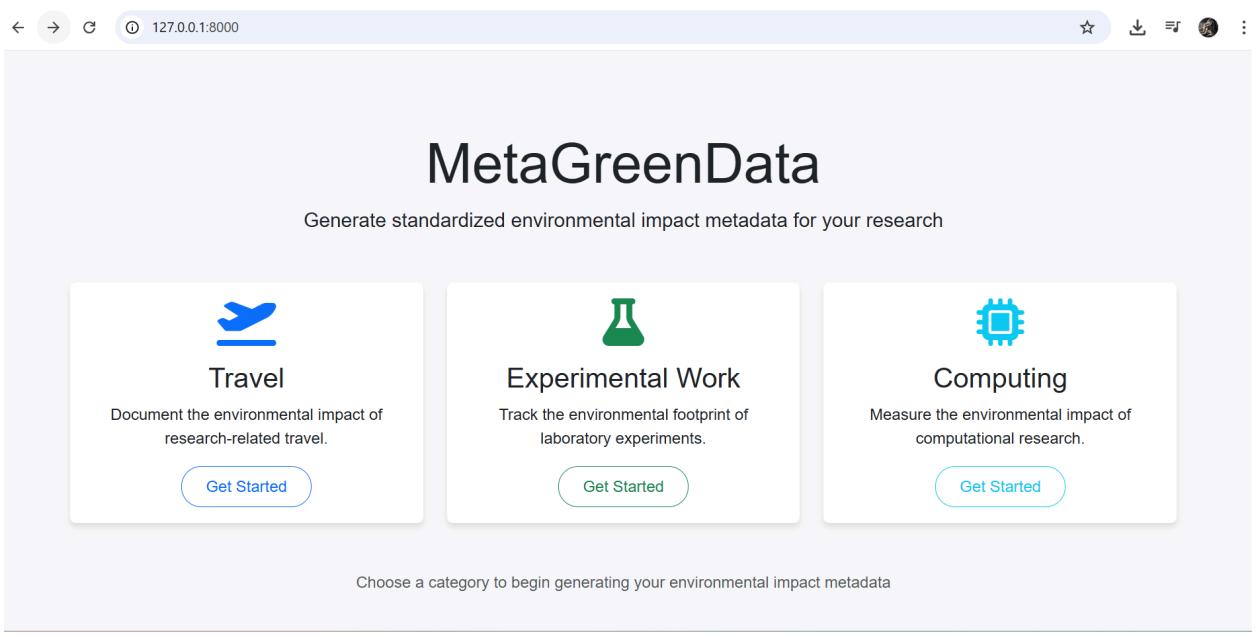
```
Downloading PyYAML-6.0.2-cp313-cp313-win_amd64.whl (156 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, PyYAML, asgiref, Django
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1/5 [sqlparse]  WARNING: The script sqlformat.exe is installed in 'C:\Users\POSH TOUCH\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4/5 [Django]  WARNING: The script django-admin.exe is installed in 'C:\Users\POSH TOUCH\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed Django-5.2.1 PyYAML-6.0.2 asgiref-3.8.1 sqlparse-0.5.3 tzdata-2025.3

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\POSH TOUCH\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\POSH TOUCH\Desktop\GreenMetaData> python3 manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
```

```
Django version 5.2.1, using settings 'metagreendata.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[12/Mar/2026 21:31:36] "GET / HTTP/1.1" 200 7187
Not Found: /favicon.ico
[12/Mar/2026 21:31:37] "GET /favicon.ico HTTP/1.1" 404 2731
[12/Mar/2026 21:32:00] "GET /computing/ HTTP/1.1" 200 48039
[12/Mar/2026 21:32:00] "GET /computing/yml-preview/?csrfmiddlewaretoken=Iu7bYQ1iDi6gTL2SLmohFhy83MMtTunC1FxHzeNUfxosuXKvbh1d92aqIzcPMe6Z&title=&repository_url=&description=&keywords=&carbon_footprint=&carbon_footprint_source=&depletion_abiotic=&depletion_abiotic_source=&particulate_matter=&particulate_matter_source=&acidification_potential=&acidification_potential_source=&ionising_radiation=&ionising_radiation_source=&photochemical_ozone=&photochemical_ozone_source=&abiotic_depletion_fossil=&abiotic_depletion_fossil_source=&freshwater_ecotoxicity=&freshwater_ecotoxicity_source=&details_of_hardware=&pue_value=&pue_method=&wue_value=&wue_method=&electrical_carbon_intensity_source= HTTP/1.1" 200 640
```



Before starting the report, I created my [github repository](#) for the project.

After filling in the fields, the report was downloaded and uploaded to my repo.

Step 5: Recommending Redesign for Improvement (with Green Software Pattern identification)

These are my redesign ideas to make the activation/latent collection + linear probing workflow in the InterPot project more eco-friendly.

1. **Batch the inference loop:**
   Right now the code processes one protein sequence at a time in the for-loop which is super inefficient on GPU (can be observed by the substantially larger GPU energy consumption in the “ESM & SAE inference” block. This can be changed to batch 8–32 sequences at once.

2. **Cache activations to disk after first run:**
   After seq_esm_acts and seq_sae_acts are computed, they can be saved with torch.save() or safetensors. This way, the probing will be just loaded from disk on the next run instead of re-running thousands of forwards.

3. **Quantizing or Distilling the ESM-2 model:**
   The ESM-2 model can be compressed via quantization post-training, which will significantly reduce the energy consumption. Alternatively, distillation can also be explored which is increasingly being implemented on protein language models (Wijaya, 2026). This suggestion aligns with the following green software pattern: Use energy efficient AI/ML models.

Limitation of this Audit and Future Improvements

This audit solely analyzed the energy consumption of an energy hotspot in this project. In the future, a more thorough measurement can be gained by measuring the consumption of the entire project (i.e each notebook, from development to testing). More elaborate breakdowns and optimization techniques can be gained through this.

Additionally, the sustainable code redesign recommendations can be implemented, and the change in the carbon footprint can be compared.